

Spike-driven Sparse Binary Representation Learning for Large-scale Multi-objective Optimization

Fei Ming, *Member, IEEE*, Xinming Shi, *Member, IEEE*, Wenxuan Pan, *Member, IEEE*,
and Yaochu Jin, *Fellow, IEEE*

Abstract—Many real-world multi-objective optimization problems contain binary-encoding and large-scale decision variables but sparse Pareto solutions, posing stiff challenges to evolutionary algorithms. With the rapid developments and broad applications of deep neural networks (DNNs), DNN-based decision space reduction has attracted increasing attention in this area. Nevertheless, the computational overhead of DNNs is a lasting challenge, and existing methods struggle to learn sparse binary representations as their continuous activations and gradient-based updates may inherently mismatch. Differently, as the third generation of artificial neural networks, spiking neural networks (SNNs) offer a more natural and efficient framework for representing sparse binary data owing to their spike-triggered event-driven working mechanism. This paper presents a first attempt to introduce spike-driven sparse binary representation learning into sparse large-scale multi-objective optimization problems (SLMOPs). Specifically, a lightweight SNN with leaky integrate-and-fire neurons is developed for evolutionary online solution modeling in solving SLMOPs. In the training process, we integrate Hebbian learning to learn the patterns of non-zero variables as local plasticity, and Kullback-Leibler divergence loss to learn the distribution of obtained Pareto solutions as global plasticity. Moreover, an adaptive firing threshold mechanism is proposed to improve the versatility across problems with different sparsities and avoid parameter tuning. Experimental results demonstrate the advantage of our methods over algorithms tailored for SLMOPs on benchmark and 20 real-world problems from different areas in terms of both effectiveness and efficiency, especially on real-world problems with unknown characteristics and sparsities.

Index Terms—Sparse large-scale multi-objective optimization, binary encoding, evolutionary algorithms, spiking neural network, spike-driven representation learning.

I. INTRODUCTION

BINARY optimization problems (BOPs) constitute one of the most fundamental classes of combinatorial optimization problems, as they are both straightforward to formulate and easy to interpret. BOPs are important since, according to [1], the 21 classical NP-complete problems identified

by Karp in combinatorial optimization can be reformulated as BOPs. In both scientific research [2]–[4] and real-world applications such as medical data analysis [5], acceleration on memristive hardware [6], and quantum computing [7], [8], BOPs or continuous optimization problems with multiple objectives, large-scale decision variables, and sparse Pareto-optimal solutions are ubiquitous, named sparse large-scale multi-objective optimization problems (SLMOPs). Also, as indicated in [9], continuous SLMOPs can be solved by maintaining a sparse binary mask along with decision variables to achieve sparsity. Despite their prevalence, SLMOPs are still challenging due to the exponential growth of the search space and the sparsity of Pareto-optimal solutions.

Typically, SLMOPs are solved by multi-objective evolutionary algorithms (MOEAs) tailored with special operators, decision space reduction, or decision variable grouping [10]. Recent years have seen rapid developments and broad applications of deep neural networks (DNNs) [11], [12]. Owing to their strong representation learning capability, DNNs are particularly effective in modeling high-dimensional, nonlinear, and complex data, enabling the extraction of informative latent structures from raw inputs. Consequently, researchers have started to develop DNN-assisted MOEAs for different purposes in MOEAs, such as Pareto set learning [13], [14], automated heuristic/MOEA design [15], [16], and neural multi-objective combinatorial optimization [17], [18]. For SLMOPs, some recent work has also been proposed for decision space reduction by DNNs. The most relevant ones include MOEA/PSL [19] and NNDREA [20]. In MOEA/PSL, a Restricted Boltzmann machine (RBM) is used to learn the representation of binary vectors in Pareto solutions at each generation during evolution to reduce dimensionality. Recently, in NNDREA, they proposed another paradigm that uses a DNN to map features of each item of the problem to binary selection decisions, reformulating an SLMOP as a low-dimensional continuous weight optimization problem.

Nonetheless, a long-lasting challenge of DNN-based methods is their computational overhead, which may lead to insufficient algorithmic efficiency especially for large-scale problems. Moreover, the training and inference mechanisms of these DNNs rely on continuous weights and gradient-based updates, which are inherently mismatched with the sparse binary nature of SLMOP data and thereby may limit their representational capacity for binary structures.

Spiking neural networks (SNNs) [21], as the third generation of artificial neural networks, encode and process information through discrete spike events, closely mimicking the

This work was supported by an International Collaboration Fund for Creative Research of National Science Foundation of China (NSFC ICFCRT) under Grant No. W2441019, a National Natural Science Foundation of China under Grant No. 62136003, and a Zhejiang Provincial Natural Science Foundation of China under Grant No. LD25F020006. (*Corresponding author: Yaochu Jin*)

Fei Ming, Wenxuan Pan, and Yaochu Jin are with the Trustworthy and General Artificial Intelligence Laboratory, School of Engineering, Westlake University, Hangzhou 310030, China. (Email: {mingfei; panwenxuan; jinyaochu}@westlake.edu.cn)

Xinming Shi is with the School of Electronics, Electrical Engineering and Computer Science, Queen's University Belfast, Belfast BT9 5BN, UK. (Email: x.shi@qub.ac.uk)

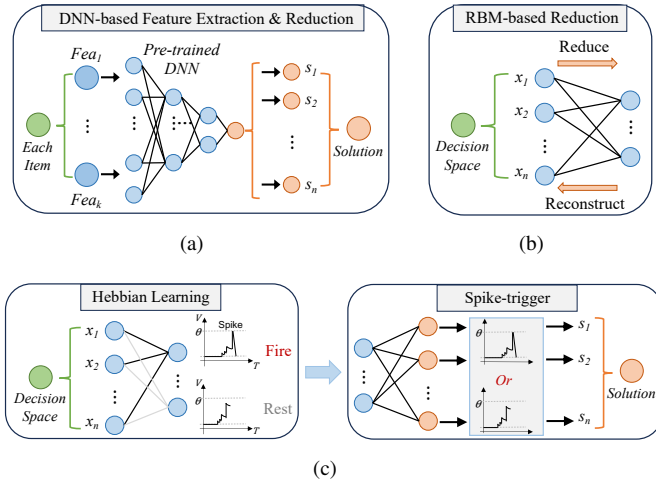


Fig. 1. Working principles of (a). NNDREA [20], (b). MOEA/PSL [19], and (c). SNN in this paper for solving binary SLMOPs.

dynamics of biological neurons [22]. Owing to their event-driven and sparse binary activation mechanisms, SNNs exhibit strong potential for temporal and spatial efficiency, as computation and communication are triggered only by spikes [23]. Importantly, the sparse and binary nature of spikes makes SNNs inherently well-suited for learning sparse binary data representations, because spike emission driven by accumulated current is inherently a 0/1 event and can also implicitly learn whether a variable takes a sparse value of non-zero reflected in the approximated Pareto solutions.

To clearly compare these different DNNs to SNNs, Fig. 1 illustrates different working principles of NNDREA (a), MOEA/PSL (b), and SNNs (c). In Fig. 1. (a), a DNN is pre-trained in an offline manner using the data of the problem. Each item of the problem (e.g., each item in the knapsack problem and each instance in the instance selection problem) then undergoes a feature extraction process to obtain the features. Thereafter, whether this item is selected is determined by the DNN using the features. In Fig. 1. (b), an RBM is devised for online binary representation learning to assist the reproduction of an MOEA. Specifically, the *Reduce* process reduces the dimensionality to a latent space where the evolutionary operators are performed to generate offspring. Then, the *Reconstruct* process restores the dimensionality to that of the original problem. In RBM, the sigmoid function is used to map the linear response to a probability in $[0,1]$; then, Bernoulli sampling is performed by comparing this probability with a uniformly distributed random number, resulting in a binary state. Differently, if SNN is adopted, as shown in Fig. 1. (c), it learns binary representations by integrating leaky membrane dynamics and spike firings. During the training process, only the weights associated with sparse fired spikes are activated and updated to develop plasticity [24], so the computational overhead can be reduced. In the reconstruction process, the spikes determine which variables should be non-zero. Thereby, the learned representations are inherently binary, sparse, and event-driven.

Based on the above idea, this paper proposes spike-driven

sparse binary representation learning, which, to the best of our knowledge, is the first attempt to apply SNN to solve SLMOPs. The main contributions are as follows:

- 1) We develop a SNN-based learning framework for evolutionary online solution modeling in solving SLMOPs. A lightweight architecture is designed to tackle few-sample evolutionary online optimization scenarios without relying on explicit label supervision. Moreover, it introduces leaky integrate-and-fire (LIF) spike-triggered dimension reduction and reconstruction to directly capture sparse binary non-zero events in SLMOPs.
- 2) In the training process, we propose to learn local plasticity by LIF-based Hebbian learning [25] and global plasticity by Kullback–Leibler (KL) divergence loss backpropagation. The former adopts the *fire together, wire together* idea to learn which variables tend to occur together among Pareto solutions in large-scale search space. The latter regulates the data distribution of reconstructed solutions to preserve the structural characteristics of the obtain Pareto solutions and facilitates the generation of promising candidate solutions.
- 3) An adaptive firing threshold mechanism is designed for the SNN to avoid manual parameter tuning and improve its versatility on SLMOPs with different sparsities. In the training of SNN at each generation, the threshold is self-learned in the training process by gradually aligning to the observed sparsity, and then used in reconstruction to automatically trigger spikes that lead to non-zero variables.

Experiments on both benchmark problems and 20 real-world problems from six different areas show that, compared to peer algorithms tailored for SLMOPs, our methods obtain competitive performance on benchmark problems and significantly better performance on different real-world problems with unknown characteristics and sparsities.

The remainder of this paper is organized as follows. Section II includes background and literature review. Section III presents the proposed methods. Then, the experimental settings are reported in Section IV, followed by the results and analyses in Section V. Finally, Section VI concludes this paper.

II. BACKGROUND AND LITERATURE REVIEW

A. Background

Without loss of generality, an SLMOP can be mathematically formulated as:

$$\begin{aligned} & \text{Minimize} && \mathbf{F}(\mathbf{x}) = (f_1(\mathbf{x}), \dots, f_m(\mathbf{x}))^T, \\ & \text{subject to} && \mathbf{x} \in \Omega^n \end{aligned} \quad (1)$$

where m is the number of objective functions; $\mathbf{x} = (x_1, \dots, x_n)^T$ is the decision vector with n decision variables (discrete or continuous); $\mathbf{x} \in \Omega^n$ and $\mathbf{F} \in \Omega^m$ are the search spaces (objective space and decision space). For real-variable SLMOPs, $\Omega^n \in \mathbb{R}^n$; while for binary-variable SLMOPs, $\Omega^n \in \{0, 1\}^n$. For SLMOPs, the number of decision variables n is typically greater than 100 and can reach up to tens or hundreds of thousands. Moreover, most decision variables in a Pareto-optimal solution are zero.

SLMOPs widely exist in different real-world applications [10]. As AI continues to be applied in a wide range of industries, the scale of both the data and problems is growing dramatically. Therefore, many optimization problems are expected to evolve into SLMOPs with multiple objectives, large-scale variables, and sparse Pareto solutions [20]. For example, in knapsack problems arising in resource allocation, portfolio selection, and system design, the number of candidate items can be hundreds or even thousands, while only a small subset is desired in Pareto optimal solutions due to capacity constraints. Similarly, in feature selection problems [5], [26], high-dimensional datasets often contain thousands or even millions of features, but only very small subsets are truly relevant for achieving multiple conflicting objectives such as accuracy and subset size. Also, in neural network training problems [3], [6], the resulting optimization formulations naturally exhibit large-scale decision spaces and sparsity.

B. DNN-assisted MOEAs for SLMOPs

Research on sparse large-scale MOEAs (SLMOEAs) has attracted significant attention in recent years; a comprehensive review has been recently provided in [10]. To conserve space and focus on the topic of this work, please refer to it for other categories of SLMOEAs. In this part, we only review DNN-assisted SLMOEAs, which is a promising yet underdeveloped research topic in this area owing to the rapid developments and broad applications of DNNs.

Tian *et al.* [19], to the best of our knowledge, first proposed to address SLMOPs by introducing unsupervised neural networks to learn low-dimensional latent representations of decision variables, thereby identifying and filtering irrelevant variables in offspring reproduction. This DNN-assisted dimensionality reduction significantly reduced the effective search space and improved optimization efficiency. Building upon this framework, an improved algorithm was proposed by Geng *et al.* [27] to further refine the unsupervised neural network by enhancing representation learning and variable selection mechanisms, leading to more accurate detection of sparse decision structures. For binary decision variables, both works employ RBM as the DNN model, which introduces several potential limitations. First, RBM training typically relies on full weight updates through iterative sampling-based contrastive divergence, resulting in substantial computational overhead when applied to large-scale decision spaces. Second, during dimensionality reduction and reconstruction, RBMs convert continuous activation probabilities into binary variables via stochastic comparisons with random numbers, which may cause information loss and reduce the consistency and stability of the reconstructed solutions.

Gao *et al.* [28] integrated deep reinforcement learning (DRL) into evolutionary algorithms to guide dimensionality reduction. By mining sparse non-zero variables through an actor-critic framework, the DRL-assisted approach dynamically focuses the search on promising subspaces whose dimensionality is much lower than the original space, yielding efficiency and robustness. Despite its effectiveness, the DRL-assisted framework exhibits several limitations. First, the reliance on

actor-critic architecture introduces considerable computational overhead, as policy evaluation and value function updates must be repeatedly performed during the evolutionary process, resulting in time consumption when scaling to large-scale decision spaces. Second, actor-critic methods are primarily designed for learning policies in continuous or smoothly parameterized action spaces, which limits their ability to effectively capture the intrinsic discreteness and sparsity of binary decision representations.

In [20], DNN-based dimensionality reduction has also been explored for SLMOPs with millions of variables, where a pre-trained DNN model learns to extract features of each item of the problem and predict whether this item should be selected. Thereby, it enables effective optimization by drastically reducing dimensionality while maintaining essential structural information of problem items. Such a paradigm is similar to end-to-end problem solving in neural combinatorial optimization, which introduces significant efficiency improvements. However, it relies on high-quality data of the problem and manually designed feature extraction to pre-train the model, which limits its applicability to scenarios where sufficient training samples are available and may reduce its generalization capability when faced with unseen or differently structured problem instances.

Shao *et al.* [29] developed a deep reinforcement learning-based guiding vector selection method, where a DNN is used to learn the mapping between population states and guiding vector rewards, enabling adaptive offspring generation during evolution. However, the method still relies on dense continuous representations and DNN-based policy learning, which may incur high computational overhead and does not explicitly model the sparse binary characteristics of Pareto solutions. Recently, they also proposed a knowledge learning-based dimensionality reduction framework [30], where a multilayer perceptron (MLP) is introduced to learn the relationship between sparse evolutionary variables and candidate dimensionality reduction schemes. By accumulating knowledge during the evolutionary process, the MLP-based model adaptively recommends suitable dimensionality reduction strategies for different evolutionary stages to balance between exploration and exploitation and enhance the adaptability of evolutionary optimization under complex sparse search spaces. However, the MLP still depends on dense continuous representations and gradient-based learning, which may incur considerable computational overhead in high-dimensional sparse search spaces. Moreover, the sparse binary characteristics of Pareto solutions are not explicitly exploited, potentially limiting the representation efficiency and adaptability of the learned model in complex sparse evolutionary scenarios.

Therefore, the following inherent limitations can be summarized for existing DNN-based MOEAs for SLMOPs. First, DNN-based frameworks introduce substantial computational overhead. Second, the continuous activation mechanisms adopted by conventional DNNs are intrinsically mismatched with sparse binary data structures.

These limitations indicate that existing DNN-based approaches are not well suited for efficiently and robustly learning sparse binary representations in evolutionary optimization,

motivating the exploration of alternative neural paradigms that are both computationally efficient and inherently compatible with sparse binary data.

C. Spiking Neural Networks

SNNs are the third generation of neural network models, where information is encoded and transmitted through discrete spike events that more closely resemble biological neural activity than conventional artificial neural networks [22], [23]. Compared with the second generation that relies on continuous activations, synchronous computation, and gradient-based weight updates, SNNs process information through discrete spike events in time. This event-driven and asynchronous computation endows SNNs with inherent temporal representation capability and the potential for computational efficiency [31]. Owing to their temporal dynamics and sparse, event-driven activations, SNNs have attracted growing interest for both neuromorphic computing research and engineering applications [32]–[35].

In the current era of large-scale AI, massive amounts of unlabeled data are continuously generated. Thereby, self-supervised and unsupervised SNNs have been attracting increasing attention for autonomous representation learning, improving generalization, and efficiently adapting to complex real-world environments without relying on extensive labeled data. Early in 2017, Roy *et al.* [36] explored unsupervised learning in SNNs where an online unsupervised structural plasticity algorithm enables SNNs to dynamically reconfigure synaptic connectivity during learning, improving adaptability and representation efficiency. Paredes-Vallés *et al.* [37] developed hierarchical unsupervised SNNs for optical flow estimation, demonstrating the capability of event-driven architectures to extract global motion perception from raw neuromorphic events. In [38], Zhang *et al.* found that spike timing-based unsupervised learning can form robust representations of orientation, disparity, and motion by leveraging precise temporal correlations among spikes. Intrinsic plasticity combined with soft-reset spiking neuron models further supports stable online unsupervised learning by regulating neuronal excitability. More recently, Pokala *et al.* [39] demonstrated that lightweight SNNs with label-free learning architectures can achieve effective multivariate temporal pattern classification and multichannel spike sorting with high energy efficiency.

Naturally, the binary and sparse characteristics of spike firing align with the characteristics of large-scale sparse binary data commonly encountered in solving SLMOPs. Consequently, SNNs are particularly well-suited for representing and learning from such data. Yet, to the best of our knowledge, no SNN-based model has been explicitly designed for solving SLMOPs, nor have SNNs been incorporated into the framework of SLMOEAs. Therefore, based on the above considerations, this work proposes the first SNN for SLMOPs.

III. PROPOSED METHODS

This section adopts a top-down manner to present the proposed methods. The overall framework is presented in

Section III-A, followed by the network architecture in Section III-B. Then, the detailed training process is provided in Section III-C, and the SNN-assisted reproduction is illustrated in Section III-D. Finally, computational complexities of our SNN and RBM are discussed in Section III-E.

A. Overall Framework

The overall framework of our proposed methods embedded into an MOEA, named MOEA-SNN, is presented in Fig. 2. Our methods mainly include the SNN training process and the SNN-assisted offspring generation in the reproduction process, which are marked in blue shade and green shade, respectively. The SNN architecture and training are detailed in Section III-B and III-C, while the SNN-assisted reproduction is detailed in Section III-D. It is important to note that the proposed SNN belongs to the online learning paradigm.

MOEA-SNN starts with the initialization and evaluation of a population with N_p solutions by the MOEA. Afterward, a parameter ρ is also initialized based on the same method in MOEA/PSL [19], which is then used to control the proportions of offspring generated by different methods. In each iteration of the evolution, the SNN is trained using the Pareto solutions in the population. Next, the mating pool is selected based on the MOEA, which contains N_p parents. This mating pool is then split into two parts, based on which two offspring generation processes are conducted simultaneously. $(1 - \rho) \times N_p$ parents are used to generate offspring by evolutionary operators in the MOEA, while $\rho \times N_p$ parents are used to generate offspring assisted by SNN. Offspring generated in the original decision space using evolutionary operators aim to preserve exploration in the original problem space and ensure the validity of the training data for SNN. The adaptive proportion mechanism in MOEA/PSL is adopted here because this mechanism can dynamically adjust the offspring generation proportion in the next generation based on the survival rates of offspring produced in different spaces. For example, if more offspring generated in the latent space survive at this generation, a higher proportion of offspring in the next generation will be produced from the latent space. Subsequently, the environmental selection of the MOEA is conducted to select the population for the next generation. Finally, when function evaluations are exhausted, the final population is output. It should be noted that, in this paper, we use the NSGA-II [40] algorithm as the basic MOEA.

Following [19], the proposed framework incorporates a control parameter ρ , which dynamically balances between SNN-assisted operator and evolutionary operator during the evolutionary process. This mechanism ensures that the SNN module is activated and contributes more effectively only when the evolutionary state is suitable for using SNN-assisted offspring generation. In the early stage of evolution and also other stages during the evolution, when the SNN-assisted offspring generation is not useful due to the overly high sparsity or immature population distribution, the framework can automatically reduce the reliance on SNN-generated offspring and allocate more computational resources to conventional evolutionary operators. This adaptive allocation helps maintain sufficient exploration capability and prevents premature

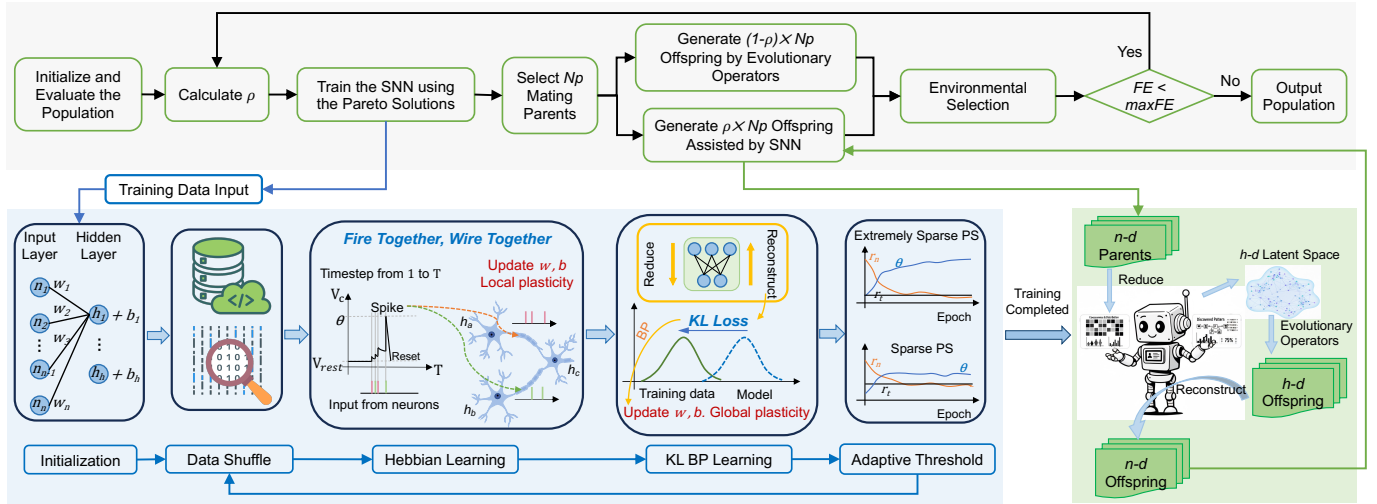


Fig. 2. The overall framework of the proposed MOEA-SNN. The top row depicts the basic workflow of an MOEA, into which the SNN training, offspring generation in the original space, and SNN-assisted offspring generation are integrated. The blue-shaded section in the lower left provides a detailed illustration of the SNN training procedure (Section III-B and III-C), while the green-shaded section in the lower right presents a detailed illustration of the SNN-based offspring generation (Section III-D).

Algorithm 1 The Framework of MOEA-SNN

Require: N_p (Population Size), $maxFE$ (Maximum Function Evaluations)

Output: $\mathcal{P}$ (Final Solution Set)

```

1:  $\mathcal{P}, \mathcal{D} \leftarrow$  Initialize a population with its corresponding decision vectors by the MOEA;
2:  $FE \leftarrow N_p$ ;
3:  $\rho \leftarrow 0.5$ ; {Proportion of SNN-based reproduction}
4: while  $FE < maxFE$  do
5:    $\mathbf{M} \leftarrow$  Train the SNN model with  $\mathcal{D}$  based on Algorithm 2;
6:    $\mathcal{M} \leftarrow$  Mating selection based on the MOEA;
7:    $\mathcal{O}_1 \leftarrow$  Generate  $\rho \times N_p$  offspring using variation operator in the latent space learned by  $\mathbf{M}$  based on Algorithm 3;
8:    $\mathcal{O}_2 \leftarrow$  Generate  $(1-\rho) \times N_p$  offspring using variation operator in the original space;
9:    $\mathcal{P}, sRatio \leftarrow$  Environmental selection on  $\mathcal{P} \cup \mathcal{O}_1 \cup \mathcal{O}_2$  based on the MOEA;
10:   $\rho \leftarrow (\rho + sRatio)/2$ ;
11:   $FE \leftarrow FE + |\mathcal{O}_1 \cup \mathcal{O}_2|$ ;
12:  if  $FE \geq maxFE$  for ML problems then
13:    Re-evaluate the population on the testing set;
14:    Perform non-dominated sorting;
15:     $\mathcal{P} \leftarrow$  Extract non-dominated solutions;
16:  end if
17: end while
18: return  $\mathcal{P}$ 

```

convergence. When the contribution of SNN-based offspring generation increases, the number of offspring generated by SNN-assisted operator will increase and the learned sparse representation can assist the search process. In this way, the SNN-assisted offspring generation is guided by the population-level evolutionary dynamics, maintaining sufficient exploration capability and preventing premature convergence by overly high reliance on SNN.

The pseudocode of MOEA-SNN is provided in Algorithm 1. The inputs include the population size N_p and the maximum function evaluations $maxFE$. The output is the final solution set in the population. The first step of the algorithm is

initialization of the population $\mathcal{P}$ as well as its corresponding decision vectors $\mathcal{D}$ (line 1). The current function evaluation FE is set to N_p (line 2), and the parameter ρ is initially set to 0.5, meaning half-and-half at the beginning (line 3). Then, the algorithm enters the main loop (lines 4-17).

In each iteration, the SNN model $\mathbf{M}$ is first trained based on Algorithm 2 using the Pareto solutions in $\mathcal{P}$ (line 5). Next, the mating selection of the MOEA is conducted to select the mating pool $\mathcal{M}$ (line 6). Based on the first $\rho \times N_p$ parents in the mating pool and $\mathbf{M}$, Algorithm 3 is performed to generate the first part of offspring (line 7). Simultaneously, the remaining mating parents are used to generate offspring based on evolutionary operators of the MOEA in the original space (line 8). Afterward, the environmental selection of the MOEA is performed to determine the population for the next generation among $\mathcal{P} \cup \mathcal{O}_1 \cup \mathcal{O}_2$, and the survival ratio of $\mathcal{O}_1$ is calculated (line 9). Thereafter, ρ is updated based on $sRatio$ (line 10). The number of function evaluations FE is updated (line 11). If the algorithm is used to solve machine learning or deep learning problems, then the population should be re-evaluated on the test dataset, and the non-dominated solutions are selected as the Pareto set (lines 12-16). Finally, the final solution set $\mathcal{P}$ is output (line 18).

B. Network Architecture

The network architecture of our proposed SNN is illustrated in Fig. 3. The input for this SNN is the decision vector of each solution with n variables. The SNN consists of an input layer and a hidden layer, which is a very simple and lightweight architecture. This design is motivated by the online learning mechanism, under which the evolutionary algorithm can provide only a limited amount of high-quality training data; consequently, overly complex network architectures may lead to overfitting. The input layer contains n neurons to handle all decision variables. The hidden layer contains a variable number of neurons, which is determined by the

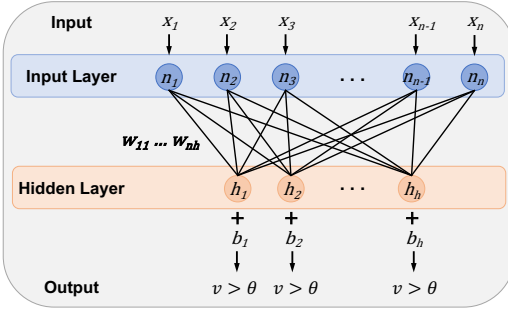


Fig. 3. The architecture of the proposed lightweight SNN.

average sparsity of the Pareto solutions in the current population. Connections exist between layers but within layers, with corresponding weights. Each neuron in the hidden layer contains a bias.

In the forward pass, the SNN compresses the dimensionality from n to an adaptive h , enabling representation learning in the latent space; in the reverse inference process, the dimensionality is reconstructed to n , transforming the solutions back to the original decision space. Thereby, through the reduction process, the search space can be significantly compressed; then, evolutionary operators can be conducted in this much smaller space. Building upon the reconstruction process, these solutions can be reconstructed into the original space as solutions for the large-scale problems.

C. Training Procedure and Learning Dynamics

The training of the proposed SNN, as illustrated in the blue-shaded part in Fig. 2, mainly contains Hebbian learning, KL loss backpropagation, and adaptive threshold processes. Given the input samples $X \in \mathbb{B}^{N \times n}$, the proposed SNN model learns through the following stages within each epoch.

1) *LIF Neuron Model*: The synaptic current is calculated by

$$I = XW + b, \quad (2)$$

and the membrane potential is updated through

$$V_t = \alpha V_{t-1} + I, \quad (3)$$

where $W \in \mathbb{R}^{n \times h}$ and $b \in \mathbb{R}^{1 \times h}$ denote the synaptic weights and biases, respectively, and $\alpha \in (0, 1)$ is the membrane potential decay factor.

A spike is generated when the membrane potential exceeds the firing threshold θ :

$$S_t = \mathbb{I}(V_t > \theta), \quad (4)$$

where $\mathbb{I}(\cdot)$ is the indicator function.

When a neuron fires, its membrane potential is reset to zero:

$$V_t(S_t = 1) = 0. \quad (5)$$

2) *Hebbian Weight Update*: At each time step, the Hebbian learning rule enhances synaptic strength between co-activated neurons in the input layer:

$$\Delta W = \eta \frac{X^T S_t}{N}, \quad (6)$$

and

$$\Delta b = \eta \text{mean}(S_t), \quad (7)$$

where η is the Hebbian learning rate. The weights are then updated by

$$W \leftarrow W + \Delta W, \quad (8)$$

and the biases are updated by

$$b \leftarrow b + \Delta b. \quad (9)$$

Through the Hebbian weight update, the co-activated neurons in the input layer can be detected. Therefore, the Hebbian learning rule is capable of discovering the latent correlations as patterns among sparse non-zero variables of Pareto solutions in large-scale search spaces [41].

3) *Reconstruction Phase (Soft Probabilistic Encoding)*: After local Hebbian weight updates, the SNN performs the forward pass

$$H = \sigma(XW + b), \quad (10)$$

and the probabilistic reconstruction

$$\hat{X} = \sigma(HW^T), \quad (11)$$

where $\sigma(x) = \frac{1}{1+e^{-x}}$ denotes the sigmoid activation function. Thereafter, the reconstruction loss can be calculated.

4) *KL Reconstruction Loss Calculation*: The reconstruction objective is defined as the KL divergence between the input X and its reconstruction $\hat{X}$:

$$\mathcal{L}_{\text{KL}} = \frac{1}{N} \sum_{i=1}^N \sum_{j=1}^{n_{in}} \left[x_{ij} \log \frac{x_{ij} + \varepsilon}{\hat{x}_{ij} + \varepsilon} + (1 - x_{ij}) \log \frac{1 - x_{ij} + \varepsilon}{1 - \hat{x}_{ij} + \varepsilon} \right], \quad (12)$$

where a small constant $\varepsilon = 10^{-8}$ is introduced to ensure numerical stability.

5) *Backpropagation-based Training*: In the training, Hebbian learning provides an effective mechanism for capturing local correlations through local plasticity learning. However, Hebbian updates are inherently local and unsupervised, and thus lack a global objective to explicitly regulate the distribution of the reconstructed data. To address this limitation, we additionally introduce the reconstruction-based loss defined by the KL divergence between the original input data and the reconstructed data.

Gradients of the reconstruction loss are estimated by

$$\frac{\partial \mathcal{L}_{\text{KL}}}{\partial H} = (\hat{X} - X)W \odot H(1 - H), \quad (13)$$

where $\odot$ denotes element-wise multiplication.

The parameter gradients are calculated by

$$\nabla_W = \frac{X^T \frac{\partial \mathcal{L}_{\text{KL}}}{\partial H} + (H^T (\hat{X} - X))^T}{N}, \quad (14)$$

and

$$\nabla_b = \text{mean} \left(\frac{\partial \mathcal{L}_{\text{KL}}}{\partial H}, 1 \right). \quad (15)$$

Afterward, the weights are updated by

$$W \leftarrow W - \eta \nabla_W, \quad (16)$$

Algorithm 2 Training Procedure of the SNN

Require: $\mathbf{X}$ (Training Data), Params (Hyper-parameters)
Output: $\mathbf{M}$ (Trained Model)

```

1: for  $\text{epoch} = 1$  to  $E$  do
2:   Randomly shuffle the training samples in  $\mathbf{X}$ ;
3:   Initialize membrane potential  $V \leftarrow 0$ ;
4:   /** Spiking Dynamics for Hebbian Learning **/
5:   for  $t = 1$  to  $T$  do
6:     Compute synaptic current  $I$  by Eq. (2);
7:     Update membrane potential  $V_t$  by Eq. (3);
8:     Generate spikes  $S_t$  by Eq. (4);
9:     Reset membrane potential of fired neurons to zero by
       Eq. (5);
10:    Update weights using Hebbian rule by Eq. (8);
11:    Update bias using mean firing activity by Eq. (9);
12:  end for
13:  /** Reconstruction KL Loss **/
14:  Compute hidden probabilities  $H$  by Eq. (10);
15:  Reconstruct input  $\hat{X}$  by Eq. (11);
16:  Compute KL reconstruction loss  $\mathcal{L}_{\text{KL}}$  by Eq. (12);
17:  /** Backpropagation **/
18:  Compute hidden-layer gradient  $\partial\mathcal{L}/\partial H$  by Eq. (13);
19:  Compute parameter gradients  $\nabla_W$  and  $\nabla_b$  by Eqs. (14)
    and (15);
20:  Update weights and biases by Eqs. (16) and (17);
21:  /** Constraints and Threshold Adaptation **/
22:  Clip weights:  $W \leftarrow \text{clip}(W, -1, 1)$ ;
23:  Update firing threshold  $\theta$  by Eq. (20);
24: end for
25: return  $\mathbf{M} \leftarrow \{W, b, \theta\}$ 

```

and the biases are updated by

$$b \leftarrow b - \eta \nabla_b. \quad (17)$$

By minimizing this KL divergence via backpropagation, the SNN is encouraged to align the global statistical structure of its generated representations with that of the Pareto solutions. This hybrid learning strategy enables the SNN to simultaneously exploit local and global plasticities, leading to more stable training dynamics and improved reconstruction quality for sparse binary inputs.

6) *Adaptive Threshold Adjustment:* Firing threshold is crucial in SNNs as it determines when a neuron fires, directly impacting spike generation, network dynamics, and computational properties [42]. The firing threshold of neurons is dynamically adjusted in each epoch through the following operations. First, at the beginning of the training, the target firing rate is calculated by

$$r_t = \frac{h}{n}, \quad (18)$$

where $h = (\sum_{i=1}^K \sum_{j=1}^n x_{ij})/K$ represents the average sparsity of Pareto solutions in the population and K is the number of Pareto solutions. The current firing rate at the current epoch is obtained by

$$r_n = \text{mean}(S_t), \quad (19)$$

Then, the firing threshold is updated by

$$\theta = \theta + 0.1 (r_n - r_t). \quad (20)$$

The initial θ value is set to 0.5, which is a relatively large value considering the sparsities of most SLMOPs in real-world

scenarios. Based on 0.5, the above adaptive mechanism could bring the following advantages. First, a relatively large initial value ensures that the SNN model does not prematurely fall into an overly sparse state, which would otherwise hinder effective exploration of the variable representation space. Second, based on the average sparsity h of the Pareto solutions obtained during the evolutionary search, Eq. (20) enables θ to adapt during training such that the number of firing neurons continuously approaches r_t , thereby approximating the sparsity level of the obtained Pareto solutions to adapt to problems with different sparsities. Moreover, the above adaptive mechanism can avoid introducing a hyper-parameter that needs to be determined relying on prior knowledge, which is usually hard to access in real-world applications, or a trial-and-error process that might be expensive.

After the training process, the firing threshold θ is finally determined and will be used in the reconstruction stage. This adaptive threshold mechanism prevents excessive firing and stabilizes the overall network dynamics.

During SNN training, the threshold is adaptively updated according to Eqs. (18) to (20), which provides a gradual self-regulating mechanism for balancing exploration and sparsity. This adaptive process helps maintain sufficient diversity during early training while encouraging sparse structural representations in later stages of training.

7) *Overall Training Process:* The pseudocode of the whole training process is summarized in Algorithm 2. The inputs include the training data (Pareto solutions in the population) $\mathbf{X}$ and the hyper-parameters which will be listed in the next section. From line 1 to line 24, each step of the training process corresponds, in order, to each of the steps described above. Finally, after training, the SNN model is output with its weights, biases, and firing threshold (line 25).

After each update step, the weights are clipped to the range $[-1, 1]$. This operation is introduced to stabilize the training process and prevent uncontrolled growth of synaptic weights, which could otherwise lead to excessively high firing rates and unstable spiking dynamics [43], [44].

The proposed learning mechanism incorporates the Hebbian learning and BP in a complementary two-timescale scheme. Specifically, the Hebbian update is performed at each time step based on spike activity, which introduces fast, local, and sparse correlation-driven plasticity. This mechanism primarily encourages the emergence of sparse binary representations and captures local co-activation structures between variables. In contrast, the BP-based update is applied at the epoch level and is driven by a global KL reconstruction loss. Its role is to enforce global consistency and regulate the latent representation space. Therefore, although the Hebbian mechanism and BP both update the same parameter matrices within the same epoch, they operate at different update frequencies and serve different functional roles. The former shapes sparse feature encoding at each time step within an epoch, while the latter provides global regularization and prevents representational drift at the epoch level.

Algorithm 3 SNN-assisted Reproduction

Require: $\mathcal{M}$ (Mating Parents), $\mathbf{M}$ (SNN Model), ρ (Proportion of SNN-assisted Offspring), N_p (Population Size)
Output: $\mathcal{O}$ (Generated Offspring)
1: $\mathcal{O} \leftarrow$ Initialize an empty set as the offspring set;
2: $\mathcal{M} \leftarrow$ Get the decision vectors of the $\rho \times N_p$ parents for SNN-assisted reproduction;
3: $\mathcal{M}' \leftarrow$ Perform dimensionality reduction by $\mathbf{M}$;
4: **for** $i = 1$ to $\rho \times N_p/2$ **do**
5: $\mathbf{p}_1, \mathbf{p}_2 \leftarrow$ Select the i -th and $2i$ -th parents from $\mathcal{M}'$;
6: $\mathbf{o}_1, \mathbf{o}_2 \leftarrow$ Generate two offspring individuals in the h -d latent space by uniform crossover on $\mathbf{p}_1, \mathbf{p}_2$;
7: $\mathcal{O} \cup \{\mathbf{o}_1, \mathbf{o}_2\}$;
8: **end for**
9: $\mathcal{O} \leftarrow$ Perform reconstruction process by $\mathbf{M}$ to obtain n -d offspring in the original space;
10: $\mathcal{O} \leftarrow$ Perform bit-flip mutation on each offspring with probability $1/n$;
11: **return** $\mathcal{O}$

D. Dimensionality Reduction and Reconstruction in SNN-assisted Reproduction

As shown in the green-shaded part in Fig. 2, in the SNN-assisted reproduction, the original n -dimensional decision space is compressed into the h -dimensional latent space. Then, in the latent space, evolutionary operators are applied to generate offspring. Afterward, these h -dimensional offspring are reconstructed to n -dimensional solutions for the SLMOP.

The reduction process transforms the input data into a sparse latent space based on spike-driven reduction through

$$V = XW + b, \quad (21)$$

and

$$H = \mathbb{I}(V > \theta). \quad (22)$$

The reconstruction performs a tied-weight decoding by projecting the spike patterns back to the input space by

$$V' = HW^\top, \quad (23)$$

and using a global threshold to obtain the binary data through

$$\hat{\mathbf{X}} = \mathbb{I}(V' > \theta). \quad (24)$$

The pseudocode of the SNN-assisted reproduction is provided in Algorithm 3. First, an empty set $\mathcal{O}$ is initialized (line 1), and the $\rho \times N_p$ parents, as well as their decision vectors, are selected as the mating pool $\mathcal{M}$ (line 2). Then, $\mathcal{M}$ undergoes the reduce process to obtain the h -dimensional parents $\mathcal{M}'$ (line 3). Next, for every two parents in $\mathcal{M}'$ (line 5), two offspring are generated in the latent space (line 6). Then, the reconstruction process is conducted to transform $\mathcal{O}$ to the original space (line 9). Afterward, the bit-flip mutation is performed on each offspring with probability $1/n$ to improve gene diversity (line 10).

E. Comparative Complexity Analysis

Let N denote the number of samples, E the number of epochs, and B the mini-batch size used in RBM training.

1) *Complexity of RBM*: The RBM is trained using mini-batch contrastive divergence. For each mini-batch of size B , the dominant computational cost arises from matrix multiplications in the positive and negative phases, including XW , $HW^\top$, and the computation of weight gradients. Each of these operations has a time complexity of $\mathcal{O}(Bnh)$, while element-wise nonlinearities and sampling operations incur only linear cost and can be neglected.

Since each epoch consists of N/B mini-batches, the total time complexity per epoch is $\mathcal{O}(\frac{N}{B} \cdot Bnh) = \mathcal{O}(Nnh)$. Therefore, the overall complexity of RBM is $\mathcal{O}(ENnh)$.

During inference, both the encoding (visible-to-hidden) and reconstruction (hidden-to-visible) stages involve a single matrix multiplication, resulting in a time complexity of $\mathcal{O}(Nnh)$.

2) *Complexity of SNN*: The SNN integrates temporal spiking dynamics with Hebbian learning and gradient-based back propagation driven by reconstruction KL loss. Within each epoch, the dominant cost is incurred by the temporal spiking simulation over T discrete time steps. At each time step, synaptic current computation and Hebbian weight updates involve matrix multiplications between the input and hidden layers, leading to a per-step complexity of $\mathcal{O}(Nnh)$. Consequently, the total cost per epoch is $\mathcal{O}(TNnh)$.

The forward pass with soft probabilistic encoding and reconstruction, followed by backpropagation through the KL reconstruction loss, is also dominated by matrix multiplications between layers and therefore contributes an additional $\mathcal{O}(Nnh)$ per epoch. Lower-order terms, such as element-wise nonlinearities and divergence computations, are omitted.

Thus, the overall training complexity of SNN is $\mathcal{O}(ETNnh)$. Similar to RBM, both the reduction and reconstruction processes in SNN require a single forward pass and thus have a time complexity of $\mathcal{O}(ENnh)$.

3) *Comparison and Discussion*: Compared with RBM, whose training complexity scales as $\mathcal{O}(ENnh)$, SNN introduces an additional temporal dimension and incurs a higher theoretical complexity of $\mathcal{O}(ETNnh)$. This increased cost comes from the explicit simulation of spiking dynamics across multiple time steps.

However, a more accurate practical complexity analysis should additionally consider the sparse firing property of SNNs. Since only a small fraction of neurons emit spikes at each time step, the effective computational cost is reduced to approximately $\mathcal{O}(ETNrn h)$, where r denotes the average firing rate. In practice, this sparsity significantly reduces the actual number of multiply-accumulate operations and memory accesses. By contrast, RBM training relies on dense sampling and contrastive divergence updates, which require repeated stochastic sampling over all hidden and visible neurons. Therefore, although the asymptotic worst-case complexity of SNN appears higher due to the temporal dimension T , the practical runtime can be substantially lower because of sparse event-driven computation.

IV. EXPERIMENTAL SETTINGS

A. SLMOPs for Comparisons

In this paper, we adopt both benchmark problems with given PF and sparsity level, and real-world problems from

different areas with unknown PF and sparsity. 40 continuous problems SMOP [9] benchmark suite is used. For real-world binary problems, we select the multi-objective feature selection (MOFS) on 10 datasets, the multi-objective knapsack problem (MOKP) on four datasets, the multi-objective instance selection (MOIS) on two datasets, and the critical node detection (CN). We also select two real-world continuous problems, the portfolio optimization (PO) and the sparse signal reconstruction (SR). The MOFS problems are provided in [26], whose objectives include classification error and feature subset size. The other problems are provided in [9]. The MOKP contains two to three knapsacks, so the objectives are the value of each knapsack. The objectives of MOIS problems are the prediction error and the number of selected instances. The CN aims to minimize the ratio of deleted nodes and the normalized pairwise connectivity of the remaining network. The PO aims to minimize risk and maximize expected return. The SR aims to minimize the sparsity of the reconstructed signal and minimize the reconstruction error. For MOFS and MOIS, the final population is re-evaluated on the test set, and the final non-dominated solutions are selected. Detailed information on these problems is listed in Table S-I in the Supplementary file. It should be noted that CN, PO, MOIS, and SR are combined into the *Other* group since they contain only a few cases.

It should be noted that, in our proposed MOEA-SNN, the SNN is designed to learn binary representations, and for parts of the solution that involve continuous encoding, we adopt the Denoising Autoencoder from MOEA/PSL [19] as a complementary module. Since MOEA/PSL contains RBM for binary variables and Denoising Autoencoder for continuous variables and we adopt the ρ mechanism from MOEA/PSL, in this sense, MOEA/PSL can be viewed as a variant of our framework where the SNN is replaced by the RBM-based binary representation learner.

B. Algorithms under Comparison

Since the problems involve binary and continuous encodings and some SLMOEAs cannot handle both, the peer algorithms selected in this study include methods applicable to both binary and continuous problems, as well as algorithms specifically designed for binary or continuous domains. For both problems, SparseEA [9], TELSO [45], and MOEA/PSL [19] are compared. For continuous problems, MOEA-NZD [46] and S-NSGA-II [47] are compared. For binary problems, TS-SparseEA [48] and DKCA [49] are compared. Comparisons to MOEA/PSL can study the performance between RBM and our proposed SNN. In addition, the recently proposed DNN-assisted AGSEA [29] and KLEA [30] are also included in the comparisons.

Since NNDREA [20] relies on data of the problem and specific feature extraction approaches, it is therefore not directly applicable to most of the problems in this paper and thus not included in the comparison.

C. Parameters Settings

For continuous encoding, the SBX [50] and polynomial mutation are adopted, where the distribution index of SBX

TABLE I
HYPERPARAMETER SETTINGS IN SNN TRAINING.

Params	Value	Meaning
E	10	No. of epochs
η	0.1	Learning rate
n	n	No. of neurons in the input layer
h	h	No. of neurons in the hidden layer
α	0.95	Decay rate
θ	0.5	Initial firing threshold
T	10	No. of time steps
ε	1e-8	Numerical stability term

uses the recommended value 20. For binary encoding, the uniform crossover and bit-flip mutation are adopted. The crossover rate is set to 0.5. The mutation rate is $1/n$.

The detailed settings of population size N_p and maximum evaluations $maxFE$ for each problem are listed in Table S-I in the Supplementary file. The hyperparameters for SNN training are presented in Table I. The most important parameters h , r_t , and θ are adaptive, while the other parameters use the recommended parameters in [19].

D. Performance Indicators

For SMOP benchmark problems, the hypervolume (HV) [51] and improved inverted generation distance (IGD+) are adopted. For MOFS problems, HV and minimum classification error (MCE) are adopted. For the rest real-world problems, HV is adopted. In addition, the CPU run time is also used to study the computational consumption.

For SMOPs, the calculations of HV and IGD+ follow the steps in [9] as the true PF of benchmark problems are given. For MOIS, portfolio optimization, and sparse signal reconstruction, the reference points are also provided, so the calculation of HV also follows the steps in [9]. Since the true PF of MOFS and MOKP are unknown, the following operations are conducted to calculate the indicator values. First, the final results of all algorithms in all 30 runs are saved. Then, the non-dominated solutions are selected, and their objective vectors are saved. The ideal and nadir points of these non-dominated solutions are obtained by which the objective space is normalized. Afterward, we use (1.1,...,1.1) as the reference point for HV calculation.

E. Statistical Analysis

On each problem, each algorithm independently executes 30 times. The mean and standard deviation values of indicators are recorded. The Wilcoxon rank-sum test with a significance level of 0.05 is employed. “+”, “−”, and “ $\approx$ ” indicate that the result of another algorithm is significantly better than, significantly worse than, and statistically similar to that obtained by MOEA-SNN. The Friedman test with a Holm correction at a significance level of 0.05 is also adopted to obtain the performance rankings of different algorithms.

V. EXPERIMENTAL RESULTS AND ANALYSIS

A. Results on Benchmark Problems

TABLE II
RANKINGS BY THE FRIEDMAN TEST ON HV, IGD+, AND RUN TIME ON
BENCHMARK PROBLEMS UNDER DIFFERENT SPARSITIES.

	Sparsity = 0.1			Sparsity = 0.02		
	HV	IGD+	Run time	HV	IGD+	Run time
SparseEA [9]	3.5375	3.5250	3.3250	2.7375	2.9250	3.5375
TELSO [45]	5.5250	5.2500	4.9500	5.4625	5.3375	5.0750
MOEA-NZD [46]	2.2625	1.9000	1.6125	2.1250	1.4625	1.5000
S-NSGA-II [47]	2.3625	2.5250	1.5500	3.8750	3.8250	1.5500
MOEA/PSL [19]	4.0750	4.4000	5.9750	3.9625	4.3250	5.6500
MOEA-SNN	3.2375	3.4000	3.5875	2.8375	3.1250	3.6875

The detailed results on indicators are presented in Tables S-II to S-IV in the Supplementary file. From the results, SparseEA and MOEA-SNN exhibit half-and-half pros and cons, TELSO performs significantly worse than MOEA-SNN, MOEA-NZD and S-NSGA-II perform significantly better, and MOEA/PSL performs significantly worse than MOEA-SNN on most problems. Overall, MOEA-SNN performs better than TELSO and MOEA/PSL, similarly to SparseEA, and worse than MOEA-NZD and S-NSGA-II.

For the CPU run time comparisons, the results show the same trends as above. As the sparsities of SMOPs can be adjusted, we also conduct comparisons using smaller sparsity. The sparsity of original SMOPs is 0.1, and the smaller one is set to 0.02. The results of 0.02 sparsity are presented in Tables S-V to S-VII in the Supplementary file. When the sparsity is much smaller, SparseEA, TELSO, and MOEA-NZD obtain slightly better overall results, but S-NSGA-II and MOEA/PSL become worse on several cases.

To better analyze the results, we conduct the Friedman test on all results on benchmark problems. The rankings of different algorithms on different indicators are summarized in Table II. From the rankings of 0.1 sparsity, MOEA-SNN shows better results than MOEA/PSL, indicating the advantage of

using SNN over RBM, as the other parts of the algorithms are almost the same. Also, MOEA-SNN achieves a competitive performance compared to SparseEA and TELSO. Nevertheless, MOEA-SNN performs significantly worse than the recently proposed MOEA-NZD and S-NSGA-II. As for time consumption, it can be found that MOEA-SNN significantly outperforms MOEA/PSL, further demonstrating the efficiency advantage of SNN over RBM. When the sparsity is 0.02, our proposed MOEA-SNN, as well as SparseEA and MOEA-NZD, obtain slightly better overall performance. Meanwhile, the performance of S-NSGA-II degenerates. Still, on time consumption, MOEA-SNN outperforms RBM in MOEA/PSL, revealing the efficiency of SNN.

However, it is very important to point out that these benchmark problems exhibit highly specific sparsity and structured patterns among nonzero variables (e.g., in many SMOPs, nonzero variables are positioned contiguously in the decision vector). Such characteristics do not adequately reflect the unknown and diverse features of real-world problems, thereby limiting their effectiveness in evaluating the general applicability of algorithms. Moreover, these artificial structures may inadvertently favor certain problem-specific strategies, leading to an overestimation of their performance on such benchmarks.

Referring to the results, as the sparsity decreases, SparseEA obtains slightly better performance because its initialization can better detect the contributing variables when the number is much smaller. Also, MOEA-NZD performs better because it can more easily detect the non-zero variables through clustering when the non-zero variables are fewer and crowded in a small region in the search space. Differently, S-NSGA-II fails to detect these non-zero variables since it relies on the evolutionary search to find which variables tend to be non-zero in the population. Such a mechanism becomes harder when the sparsity increases. Even so, MOEA/PSL and MOEA-SNN

TABLE III
HV RESULTS OF MOEA-SNN AND OTHER ALGORITHMS ON REAL-WORLD PROBLEMS, INCLUDING THE MEAN VALUES AND STANDARD DEVIATIONS SHOWN IN PARENTHESES. THE HIGHLIGHTED ENTRY IN EACH ROW INDICATES THE BEST-PERFORMING ALGORITHM FOR THIS DATASET.

Dataset Name	D	SparseEA [9]	TELSO [45]	TS-SparseEA [48]	DKCA [49]	MOEA/PSL [19]	MOEA-SNN
Tumor9	5726	9.2209e-1 (6.90e-2) ≈	9.1973e-1 (2.80e-2) ≈	9.3115e-1 (4.44e-2) ≈	5.0318e-1 (1.16e-2) −	8.8959e-1 (5.35e-2) −	9.2997e-1 (1.72e-2)
TOX171	5748	9.8467e-1 (6.44e-3) −	9.8195e-1 (5.44e-3) −	9.8461e-1 (6.06e-3) −	9.1396e-1 (4.09e-3) −	9.0614e-1 (1.30e-1) ≈	9.8563e-1 (4.63e-4)
AR10P	2400	9.5323e-1 (6.06e-3) ≈	9.4300e-1 (4.91e-3) −	9.5109e-1 (1.46e-2) ≈	9.2026e-1 (9.05e-3) −	9.1311e-1 (1.02e-1) ≈	9.5371e-1 (9.40e-3)
Arrhythmia	278	8.3526e-1 (3.60e-2) +	7.7993e-1 (2.87e-2) −	8.2026e-1 (2.97e-2) ≈	8.1702e-1 (2.70e-2) ≈	8.1206e-1 (3.00e-2) −	8.1881e-1 (3.29e-2)
Brain1	5920	9.6206e-1 (3.04e-2) ≈	9.5815e-1 (1.84e-2) −	9.6588e-1 (2.09e-2) ≈	6.8335e-1 (2.72e-3) −	9.4257e-1 (7.84e-2) ≈	9.7224e-1 (1.93e-2)
Carcinom	9182	9.6335e-1 (1.59e-2) −	9.6038e-1 (8.69e-3) −	9.7366e-1 (4.05e-3) −	8.6766e-1 (4.95e-3) −	9.6038e-1 (4.21e-2) −	9.7897e-1 (2.15e-3)
Brain2	10367	9.8687e-1 (1.43e-2) −	9.8836e-1 (2.48e-3) ≈	9.8746e-1 (1.20e-2) −	9.4651e-1 (4.87e-3) −	9.5138e-1 (1.05e-1) −	9.9039e-1 (9.26e-5)
CLLSUB111	11340	9.8915e-1 (3.39e-3) −	9.8849e-1 (1.86e-3) −	9.8896e-1 (2.66e-3) −	9.6459e-1 (2.62e-3) −	9.5977e-1 (7.50e-2) ≈	9.9072e-1 (1.30e-4)
Tumor11	12533	9.6178e-1 (1.65e-2) −	9.5863e-1 (4.59e-3) −	9.6673e-1 (1.03e-2) −	8.4686e-1 (3.79e-3) −	9.4865e-1 (7.22e-2) ≈	9.7814e-1 (2.11e-3)
CNS	7129	9.8822e-1 (1.23e-3) −	9.8749e-1 (2.39e-5) −	9.8718e-1 (4.58e-3) ≈	9.5362e-1 (1.04e-4) −	9.6334e-1 (9.32e-2) ≈	9.8834e-1 (5.40e-4)
+ / − / ≈		1/6/3	0/8/2	0/5/5	0/9/1	0/4/6	
Sparse-KP-250	250	8.2126e-2 (1.79e-3) −	4.0749e-2 (3.65e-3) −	8.0407e-2 (2.22e-3) −	5.9557e-2 (4.24e-3) −	8.5290e-2 (2.19e-3) −	8.6740e-2 (1.59e-3)
Sparse-KP-500	500	7.0656e-2 (1.32e-3) −	4.0791e-2 (1.98e-3) −	6.8512e-2 (2.33e-3) −	5.3975e-2 (4.38e-3) −	7.4898e-2 (4.06e-3) −	7.8724e-2 (1.23e-3)
Sparse-KP-1000	1000	6.0775e-2 (1.95e-3) −	4.0251e-2 (1.91e-3) −	5.7077e-2 (3.49e-3) −	4.7614e-2 (4.30e-3) −	6.2544e-2 (4.62e-3) −	7.1312e-2 (1.32e-3)
Sparse-KP-2000	2000	5.0642e-2 (9.70e-4) −	3.9528e-2 (1.32e-3) −	4.7003e-2 (8.47e-4) −	4.3372e-2 (3.09e-3) −	4.9042e-2 (5.13e-3) −	6.1652e-2 (1.23e-3)
+ / − / ≈		0/4/0	0/4/0	0/4/0	0/4/0	0/4/0	
Sparse-IS	4177	6.9342e-1 (2.76e-3) −	6.8825e-1 (4.63e-3) −	7.2790e-1 (6.33e-3) −	6.9077e-1 (2.21e-3) −	7.9250e-1 (2.21e-2) ≈	7.8878e-1 (1.54e-2)
Sparse-IS	11055	9.4942e-1 (5.23e-4) −	9.4797e-1 (9.91e-4) −	9.5302e-1 (9.84e-4) −	9.4886e-1 (6.91e-4) −	9.2721e-1 (1.05e-1) ≈	9.6929e-1 (2.57e-3)
Sparse-CN	452	9.9617e-1 (1.48e-5) ≈	9.9408e-1 (4.86e-4) −	9.9608e-1 (6.62e-5) −	9.9606e-1 (4.87e-5) −	9.9599e-1 (1.75e-4) ≈	9.9614e-1 (1.04e-4)
Sparse-PO	5000	1.1870e-1 (2.60e-3) ≈	1.1900e-1 (3.29e-3) ≈	1.1212e-1 (5.57e-3) −	1.1894e-1 (3.31e-3) ≈	1.0606e-1 (1.36e-2) −	1.1615e-1 (5.81e-3)
Sparse-SR	1024	2.8306e-1 (1.33e-2) −	1.4817e-1 (1.20e-2) −	2.2786e-1 (3.99e-3) −	2.8632e-1 (1.15e-2) −	2.3764e-1 (1.77e-2) −	3.0120e-1 (1.20e-2)
Sparse-SR	4096	9.8175e-2 (4.75e-4) −	9.7780e-2 (5.25e-4) −	9.7682e-2 (7.37e-4) −	9.8134e-2 (4.57e-4) −	1.1094e-1 (1.08e-2) −	1.2942e-1 (4.28e-3)
+ / − / ≈		0/4/2	0/5/1	0/6/0	0/5/1	0/3/3	

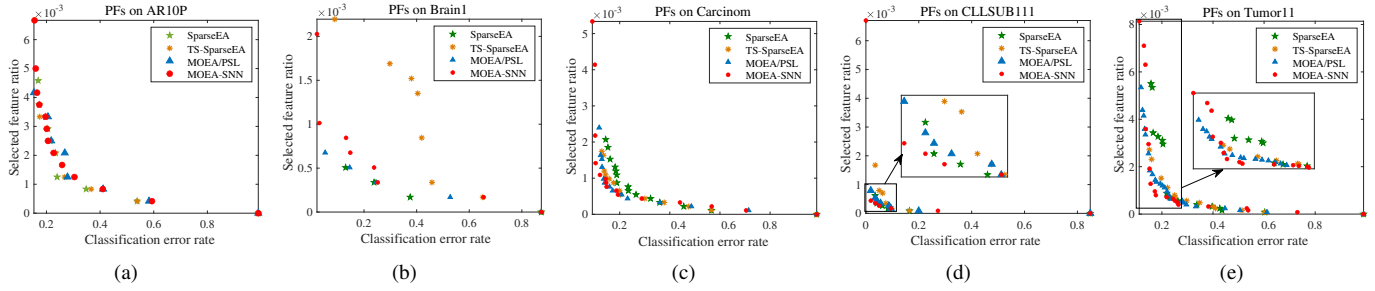


Fig. 4. The PFs of the top-ranked four algorithms on several representative MOFS problems.

TABLE IV
RANKINGS BY THE FRIEDMAN TEST ON HV AND RUN TIME ON REAL-WORLD PROBLEMS, AND ALSO THE MCE ON MOFS PROBLEMS.

	MOFS			MOKP		Other	
	HV	MCE	Run time	HV	Run time	HV	Run time
SparseEA [9]	2.50	3.40	3.30	2.75	3.25	3.10	1.80
TELSE [45]	4.00	3.85	4.50	6.00	5.00	5.00	4.80
TS-SparseEA [48]	2.40	3.00	2.30	4.00	2.00	3.60	3.60
DKCA [49]	5.50	5.80	4.40	5.00	1.25	3.10	3.00
MOEA/PSL [19]	5.25	3.50	3.90	2.25	6.00	4.00	6.00
MOEA-SNN	1.35	1.45	2.60	1.00	3.50	2.18	1.80

maintain stable performance on different sparsities, revealing that the adaptive mechanisms are effective.

To further assess the versatility and generality of different algorithms, we evaluate MOEA-SNN and the peer algorithms on real-world problems from diverse application domains with unknown sparsity patterns and heterogeneous variable interactions. The results and analyses are presented in Section V-B.

B. Results on Real-world Problems

1) *Indicator Results*: The results on HV on real-world problems are presented in Table III, while the MCE results on MOFS and run time results on all are reported in the Supplementary file. It can be determined that, on MOFS, MOEA-SNN achieves superior performance on almost all cases compared to all peer algorithms on HV and MCE. On time consumption, MOEA-SNN also significantly outperforms SparseEA, TELSE, DKCA, and MOEA/PSL. From results on MOKP, MOEA-SNN also significantly outperforms all other algorithms on all cases. As for run time, SNN is significantly more time-efficient than RBM in MOEA/PSL. However, for these relatively low-dimensional problems, TS-SparseEA and DKCA are much more efficient. The results on Other problems show that, MOEA-SNN presents better results compared to most other methods. We can also find that it is much more time-efficient than other algorithms, except that SparseEA performs better on low-dimensional CN and SR, and DKCA performs better on CN. The advantage of SNN on time consumption reveals its efficiency over most existing methods for SLMOPs, especially the DNN-based method.

The Friedman test is conducted on real-world results, and the rankings of different algorithms are summarized in Table IV. Surprisingly, MOEA-SNN achieves significantly better overall performances on all real-world problems, and its time

efficiency is also prominent, especially on relatively high-dimensional problems. Compared to benchmark results, it can be clearly found that when the problems are with unknown sparsities and patterns of non-zero variables, those algorithms performing well on the benchmark cannot yield superiority on different real-world problems. However, our proposed MOEA-SNN exhibits superiority on almost all these problems. Therefore, it is further demonstrated that the proposed MOEA-SNN is efficient and robust on different real-world SLMOPs.

Overall, compared with synthetic benchmark problems, the considered real-world problems exhibit substantially more complex and heterogeneous characteristics, possibly including irregular sparsity patterns, diverse variable interactions, and nonuniform Pareto structures. Under these challenging scenarios, the proposed MOEA-SNN demonstrates more stable and competitive performance across different application domains. These results indicate that the spike-driven sparse representation learning mechanism provides stronger adaptability and generalization capability for practical SLMOPs, which further validates the effectiveness of the proposed evolutionary online solution modeling framework in real-world scenarios.

2) *PFs Results on Real-world Cases*: To better observe the results and see the advantages of MOEA-SNN, we depict the PFs of top-ranked algorithms on some of the problems.

The PFs obtained by the top-ranked four algorithms on MOFS cases are depicted in Fig. 4. On AR10P, MOEA-SNN obtains feature subsets with higher accuracy but slightly larger feature subsets. Moreover, it obtains more solutions in the knee region. On Brain1, MOEA-SNN obtains worse solutions in the knee region but much better solutions with higher accuracy. On Carcinom, the PF of MOEA-SNN is better on the left region with higher accuracy. On CLLSUB111, MOEA-SNN finds one subset with much higher accuracy as well as subset size, and finds better knee solutions. On Tumor11, the results are similar to those on CLLSUB111.

The Pareto fronts obtained by the top-ranked three algorithms on MOIS with 11055 instances and SR are depicted in Fig. 5. On MOIS, the PF of MOEA-SNN achieves better convergence on validation error and selected instance ratio. On SR, MOEA-SNN achieves better Pareto solutions in the low-loss area because those solutions are of better sparsities than those of other algorithms.

The PFs of all algorithms on 500- and 2000-dimensional MOKPs are depicted in Fig. 6. From the PFs, MOEA-SNN is capable of searching for item combinations that dominate

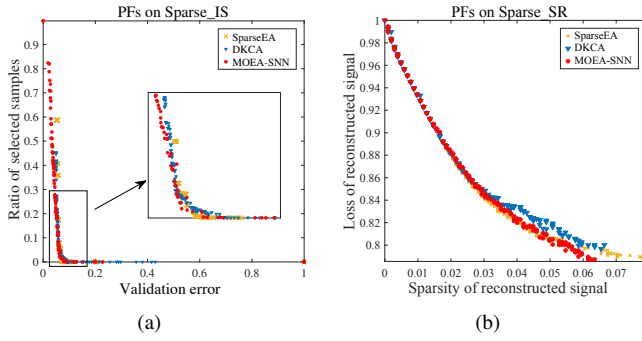


Fig. 5. The PFs of the top-ranked three algorithms on MOIS and SR.

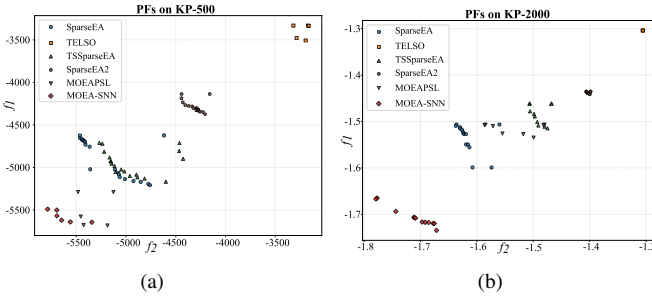


Fig. 6. The PFs of all algorithms on 500-d and 2000-d MOKP cases.

the PFs of other algorithms, leading to higher knapsack values across all knapsacks.

3) *PF Dynamics with Evolutionary Trajectory*: The dynamic changes of the Pareto front during the evolutionary process are illustrated in Fig. 7 by presenting the PFs of five MOFS datasets at four different stages: the initialization stage, after completing one-third of the total iterations, after completing two-thirds of the total iterations, and the final stage. It should be noted that all these solutions are evaluated on the training data.

The experimental results demonstrate that, from the initialization stage to the early evolutionary stage, the population rapidly converges toward promising regions. Subsequently, from the middle to the late stages, the convergence performance on all datasets, as well as diversity on Carcinom and Tumor11, is further improved.

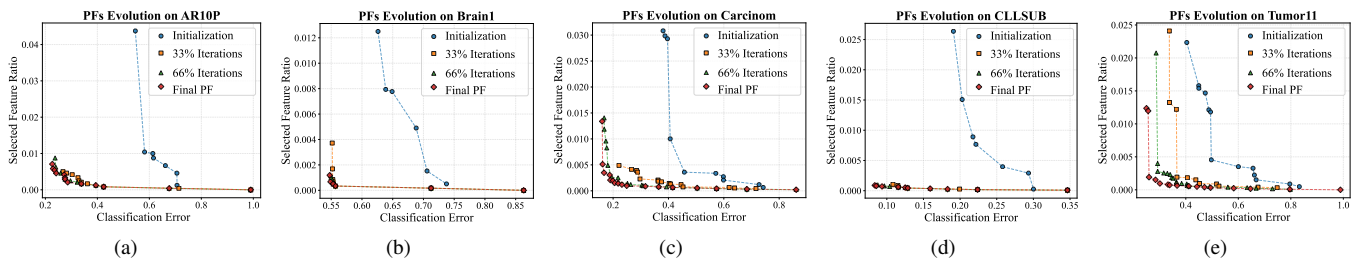


Fig. 7. The PF dynamics with the evolutionary trajectory on five MOFS cases, where the Pareto fronts of the initialized population, population at around 1/3 iterations, population at around 2/3 iterations, and the final population are depicted.

C. Ablation Studies

To verify the effectiveness of the proposed mechanisms, ablation studies are conducted with the following variants:

- **woHebbian**: This variant removes the Hebbian learning in the training of SNN to verify the necessity of Hebbian learning for local plasticity.
- **woKL**: This variant removes the KL loss backpropagation in the training of SNN to verify the necessity of it for global plasticity.
- **woEvo**: This variant removes the offspring generation in the original space to verify that it is important to incorporate original space exploration.
- **woSNN**: This variant removes the SNN-assisted offspring generation to verify the effectiveness of the proposed SNN.

The detailed results on indicators are reported in Tables S-XII to S-XVII in the Supplementary file. A summary of the Wilcoxon rank-sum results and Friedman rankings is presented in Table V.

Overall, on benchmark and real-world problems, the results verify the effectiveness of each of the proposed mechanisms. One interesting point is that, for MOKPs, the help from SNN-assisted reproduction is limited. This might be owed to the simple search spaces and fewer correlations between variables in these relatively low-dimensional MOKPs.

Since MOEA/PSL can be regarded as the variant of replacing SNN by RBM, the results from both benchmark and real-world problems demonstrate that the practical efficiency, as well as representation accuracy, of our proposed SNN is significantly better than RBM.

D. Parameter Studies on T

The number of time steps T can have an impact on the representation accuracy and computational cost of the SNN. When T increases, the time consumption also increases. However, its impact on representation effectiveness remains unknown. In this part, we conduct parameter studies on T on the real-world problems, where T is set to 5 and 20 with all other experimental settings unchanged.

The results are presented in Table S-XVIII in the Supplementary file. When T is set to 5, the $+/-/\approx$ performance is 0/2/17. When T is set to 20, the $+/-/\approx$ performance is 1/2/16. Therefore, it can be found that the value of T for training does not have a significant impact on the performance.

TABLE V
WILCOXON TEST RESULTS (+/-/≈) AND FRIEDMAN TEST RESULTS (RANKINGS) OF ABLATION STUDIES ON BENCHMARK AND DIFFERENT REAL-WORLD PROBLEMS.

	Benchmark				MOFS				MOKP		Other	
	HV		IGD+		HV		MCE		HV		HV	
	+/-/≈	Ranking	+/-/≈	Ranking	+/-/≈	Ranking	+/-/≈	Ranking	+/-/≈	Ranking	+/-/≈	Ranking
woHebbian	2/22/16	2.9750	2/22/16	3.1375	0/10/0	3.8000	0/10/0	4.5000	0/2/2	3.2500	1/3/1	3.5000
woKL	4/15/21	2.6000	4/16/20	2.5000	0/10/0	4.0000	0/10/0	4.3000	0/3/1	3.7500	1/3/1	2.9000
woEvo	0/40/0	5.0000	0/40/0	5.0000	0/3/7	2.8500	0/3/7	2.4000	0/4/0	5.0000	1/2/2	2.8000
woSNN	2/18/20	2.6500	2/19/19	2.7125	0/4/6	3.1000	0/4/6	2.7000	0/0/4	1.6000	0/4/1	3.4000
MOEA-SNN		1.7750		1.6500		1.2500		1.1000		1.4000		2.2000

TABLE VI
WILCOXON TEST RESULTS (+/-/≈) AND FRIEDMAN TEST RESULTS (RANKINGS) OF COMPARISON STUDIES WITH ADVANCED SLMOEAs.

	HV		MCE		Run Time	
	+/-/≈	Ranking	+/-/≈	Ranking	+/-/≈	Ranking
AGSEA [29]	2/16/1	2.7368	0/10/0	3.0000	6/13/0	2.3684
KLEA [30]	2/11/6	1.9211	0/7/3	2.0000	10/8/1	1.7895
MOEA-SNN		1.3421		1.0000		1.8421

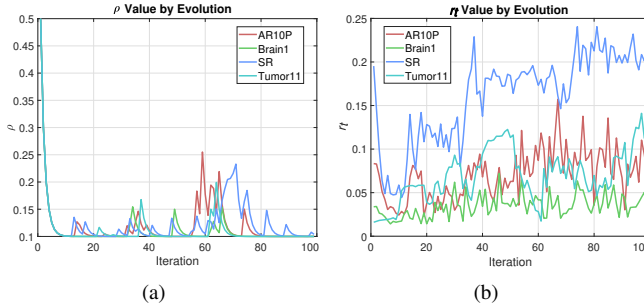


Fig. 8. The ρ values and r_t values at each iteration of MOEA-SNN in solving different problems.

However, in some cases, setting it too small may lead to a slight degradation in performance. Readers may refer to the table for parameter tuning in practical applications.

E. Comparisons with Advanced DNN-based SLMOEAs

To verify the competitiveness of the proposed MOEA-SNN against recently proposed advanced DNN-assisted SLMOEAs, in this part, we conduct comparison studies to AGSEA and KLEA. The experimental settings are all the same. The detailed results on HV, MCE (only MOFS problems), and run time are listed in Tables S-XIX to S-XXV in the Supplementary file. The statistical analyses results are provided in Table VI.

From the results on HV and MCE on MOFS, MOEA-SNN significantly outperforms these two state-of-the-art and DNN-assisted SLMOEAs, demonstrating the competitiveness of our proposed MOEA-SNN. As for the computational cost comparisons on run time, KLEA show slight advantage over MOEA-SNN while MOEA-SNN performs better than AGSEA. In conclusion, our proposed MOEA-SNN achieves superior performance and efficiency on different real-world problems.

F. Model Behaviors Studies

To investigate the behaviors of the evolutionary process and the proposed SNN model, we conduct the following two groups of experimental analyses on different real-world problems.

In the first group of experiments, we present the variations of the ρ and r_t values during the entire evolutionary process of MOEA-SNN on four different problems in Fig. 8.

The changes of ρ in Fig. 8. (a) indicate that SNN-assisted offspring generation plays a significant role only at certain critical evolutionary stages, while it consumes relatively few computational resources during most iterations. Moreover, the value of ρ varies across different problems, suggesting that the SNN-assisted offspring generation can adapt to problem-specific characteristics and automatically adjust its computational allocation accordingly.

The variations of r_t in Fig. 8. (b) intuitively demonstrate that these real-world problems exhibit substantial differences in sparsity. For example, the sparsity of Brain1 can be as low as approximately 0.02, while AR10 exhibits a much higher sparsity level up to approximately 0.25. Meanwhile, the MOEA-SNN algorithm can escape from originally high sparsity owing to the cooperation of conventional operators and the adaptive threshold mechanism.

In the second group of experiments, we plot the variations of r_t , r_n , and θ during the training process in Fig. 9, where we set the number of epochs to 20 to investigate their dynamic behavior. Fig. 9. (a) shows the results on the AR10P dataset (relatively sparse) when ρ reaches 0.18. Fig. 9. (b) presents the results on the Brain1 dataset (extremely sparse) when ρ reaches 0.22.

These two figures intuitively illustrate that the value of θ increases progressively based on r_t , starting from the initial stage, in order to reduce the firing rate, which in turn leads to the decrease in r_n . However, it is worth noting that although the overall trend of r_n is consistent with expectations, its exact values differ slightly from those anticipated in Section III. This discrepancy is likely due to the fact that, within the LIF dynamics of the SNN, r_n and r_t do not need to strictly match in order for the network to learn the desired level of sparsity.

VI. CONCLUSIONS AND FUTURE WORK

This paper presents the first work that introduces SNN into SLMOPs. The developed lightweight SNN model successfully utilizes the nature of sparse binary representation learning of

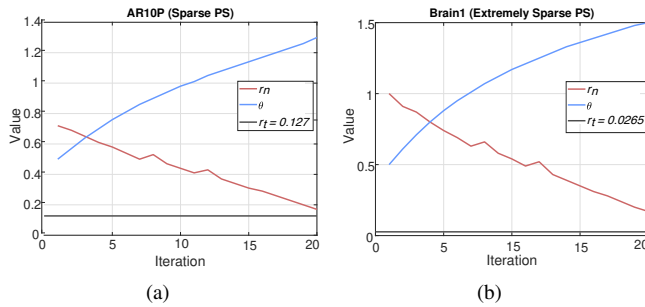


Fig. 9. The r_n , θ , and r_t values at each epoch of SNN training in solving the MOFS problem on the AR10P (a) and Brain1 (b) datasets.

SNN in SLMOEAs. For the training, we propose to integrate LIF-based Hebbian learning and KL divergence loss backpropagation to endow the model with the capabilities of learning non-zero variable patterns and regulating the Pareto optimality, respectively. The adaptive firing threshold mechanism ensures the versatility of this model to problems with different sparsities and avoids human-involved parameter tuning.

Experiments are conducted on both benchmark and 20 real-world problems from six different areas. From the results, the proposed SNN outperforms existing DNN-based algorithms in terms of both effectiveness and efficiency. The results also demonstrate that, although the proposed MOEA-SNN does not exhibit a significant advantage over the peer algorithms on benchmark problems, it is superior on 20 real-world problems with complex and unknown characteristics. We also analyze the differences between benchmark and real-world SLMOPs, highlighting the importance of experimental verifications on different real-world problems rather than only benchmark problems.

In the future, it is desired to apply MOEA-SNN to SLMOPs arising from other real-world scenarios to study its performance. Moreover, the current architecture and training procedure are very simple yet easily extensible, so many advanced and complex SNN architectures and training methods are also worth exploring [52], [53]. The performance on benchmark problems show the bottleneck in the SNN when learning such strongly correlated and highly structured patterns, indicating the scope of applicability of the proposed MOEA-SNN.

The source code can be obtained through the following GitHub link <https://github.com/mingfei123/MOEA-SNN>.

REFERENCES

- [1] A. Lucas, "Ising formulations of many np problems," *Frontiers in Physics*, vol. Volume 2 - 2014, 2014.
- [2] M. J. A. Schuetz, J. K. Brubaker, and H. G. Katzgraber, "Combinatorial optimization with physics-inspired graph neural networks," *Nature Machine Intelligence*, vol. 4, pp. 367–377, 2022.
- [3] Z. Wang, H. Hong, K. Ye, G.-E. Zhang, M. Jiang, and K. C. Tan, "Manifold interpolation for large-scale multiobjective optimization via generative adversarial networks," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 34, no. 8, pp. 4631–4645, 2023.
- [4] J. Russell, I. Gavier, D. Patel, E. Rietman, and H. Siegelmann, "Optimizing neural network representations of boolean networks," in *International Conference on Learning Representations (ICLR)*, 2025.
- [5] H. Saadatmand and M.-R. Akbarzadeh-T, "Many-objective jaccard-based evolutionary feature selection for high-dimensional imbalanced data classification," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 46, no. 12, pp. 8820–8835, 2024.
- [6] B. Lyu, S. Wang, S. Wen, K. Shi, Y. Yang, L. Zeng, and T. Huang, "Autogmap: Learning to map large-scale sparse graphs on memristive crossbars," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 35, no. 9, pp. 12 888–12 898, 2024.
- [7] M. Sciorilli, L. Borges, T. L. Patti, D. García-Martín, G. Camilo, A. Anandkumar, and L. Aolita, "Towards large-scale quantum optimization solvers with few qubits," *Nature Communications*, vol. 16, no. 1, p. 476, 2025.
- [8] A. Kotil, E. Pelofske, S. Riedmüller, D. J. Egger, S. Eidenbenz, T. Koch, and S. Woerner, "Quantum approximate multi-objective optimization," *Nature Computational Science*, vol. 5, pp. 1168–1177, 2025.
- [9] Y. Tian, X. Zhang, C. Wang, and Y. Jin, "An evolutionary algorithm for large-scale sparse multiobjective optimization problems," *IEEE Transactions on Evolutionary Computation*, vol. 24, no. 2, pp. 380–393, 2020.
- [10] S. Shao, Y. Tian, Y. Zhang, S. Yang, P. Zhang, C. He, X. Zhang, and Y. Jin, "Evolutionary computation for sparse multi-objective optimization: A survey," *ACM Comput. Surv.*, vol. 57, no. 11, Jun. 2025.
- [11] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *nature*, vol. 521, no. 7553, pp. 436–444, 2015.
- [12] H. Naveed, A. U. Khan, S. Qiu, M. Saqib, S. Anwar, M. Usman, N. Akhtar, N. Barnes, and A. Mian, "A comprehensive overview of large language models," *ACM Trans. Intell. Syst. Technol.*, vol. 16, no. 5, Aug. 2025.
- [13] Y. Lu, B. Li, and A. Zhou, "Are you concerned about limited function evaluations: Data-augmented pareto set learning for expensive multi-objective optimization," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 13, pp. 14 202–14 210, Mar. 2024.
- [14] J. Cheng, X. Lin, B. Xue, and Q. Zhang, "Parametric pareto set learning: Amortizing multi-objective optimization with parameters," *IEEE Transactions on Evolutionary Computation*, pp. 1–1, 2025, doi:10.1109/TEVC.2025.3630787.
- [15] Y. Huang, S. Wu, W. Zhang, J. Wu, L. Feng, and K. C. Tan, "Autonomous multi-objective optimization using large language model," *IEEE Transactions on Evolutionary Computation*, pp. 1–1, 2025, doi:10.1109/TEVC.2025.3561001.
- [16] S. Yao, F. Liu, X. Lin, Z. Lu, Z. Wang, and Q. Zhang, "Multi-objective evolution of heuristic using large language model," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, no. 25, pp. 27 144–27 152, Apr. 2025.
- [17] J. Chen, Z. Zhang, Z. Cao, Y. Wu, Y. Ma, T. Ye, and J. Wang, "Neural multi-objective combinatorial optimization with diversity enhancement," in *Advances in Neural Information Processing Systems*, vol. 36. Curran Associates, Inc., 2023, pp. 39 176–39 188.
- [18] J. Chen, J. Wang, Z. Cao, and Y. Wu, "Neural multi-objective combinatorial optimization via graph-image multimodal fusion," in *International Conference on Representation Learning*, 2025, pp. 33 964–33 983.
- [19] Y. Tian, C. Lu, X. Zhang, K. C. Tan, and Y. Jin, "Solving large-scale multiobjective optimization problems with sparse optimal solutions via unsupervised neural networks," *IEEE Transactions on Cybernetics*, vol. 51, no. 6, pp. 3115–3128, 2021.
- [20] Y. Tian, L. Wang, S. Yang, J. Ding, Y. Jin, and X. Zhang, "Neural network-based dimensionality reduction for large-scale binary optimization with millions of variables," *IEEE Transactions on Evolutionary Computation*, vol. 29, no. 6, pp. 2328–2342, 2025.
- [21] W. Maass, T. Natschlager, and H. Markram, "Real-time computing without stable states: A new framework for neural computation based on perturbations," *Neural computation*, vol. 14, no. 11, pp. 2531–2560, 2002.
- [22] W. Maass, "Networks of spiking neurons: The third generation of neural network models," *Neural Networks*, vol. 10, no. 9, pp. 1659–1671, 1997.
- [23] K. Roy, A. Jaiswal, and P. Panda, "Towards spike-based machine intelligence with neuromorphic computing," *Nature*, vol. 575, no. 7784, pp. 607–617, 2019.
- [24] G. G. Turrigiano, "The self-tuning neuron: synaptic scaling of excitatory synapses," *Cell*, vol. 135, no. 3, pp. 422–435, 2008.
- [25] D. O. Hebb, *The organization of behavior: A neuropsychological theory*. Psychology press, 2005.
- [26] R. Jiao, B. H. Nguyen, B. Xue, and M. Zhang, "A survey on evolutionary multiobjective feature selection in classification: Approaches, applications, and challenges," *IEEE Transactions on Evolutionary Computation*, vol. 28, no. 4, pp. 1156–1176, 2024.
- [27] H. Geng, J. Shen, Z. Zhou, and K. Xu, "An improved large-scale sparse multi-objective evolutionary algorithm using unsupervised neural network," *Applied Intelligence*, vol. 52, no. 9, pp. 10 290–10 309, 2022.
- [28] M. Gao, X. Feng, H. Yu, and X. Li, "An efficient evolutionary algorithm based on deep reinforcement learning for large-scale sparse multiobjec-

- tive optimization,” *Applied Intelligence*, vol. 53, no. 18, pp. 21 116–21 139, 2023.
- [29] S. Shao, Y. Tian, and X. Zhang, “Deep reinforcement learning assisted automated guiding vector selection for large-scale sparse multi-objective optimization,” *Swarm and Evolutionary Computation*, vol. 88, p. 101606, 2024.
- [30] S. Shao, Y. Tian, Y. Zhang, and X. Zhang, “Knowledge learning-based dimensionality reduction for solving large-scale sparse multiobjective optimization problems,” *IEEE Transactions on Cybernetics*, vol. 55, no. 7, pp. 3471–3484, 2025.
- [31] G. Li, L. Deng, H. Tang, G. Pan, Y. Tian, K. Roy, and W. Maass, “Brain-inspired computing: A systematic survey and future trends,” *Proceedings of the IEEE*, vol. 112, no. 6, pp. 544–584, 2024.
- [32] X. Shi, Z. Zeng, L. Yang, and Y. Huang, “Memristor-based circuit design for neuron with homeostatic plasticity,” *IEEE Transactions on Emerging Topics in Computational Intelligence*, vol. 2, no. 5, pp. 359–370, 2018.
- [33] Y. Yan, Q. Yang, Y. Wu, H. Liu, M. Zhang, H. Li, K. C. Tan, and J. Wu, “Efficient and robust temporal processing with neural oscillations modulated spiking neural networks,” *Nature communications*, vol. 16, no. 1, p. 8651, 2025.
- [34] L. Fan, H. Shen, X. Lian, Y. Li, M. Yao, G. Li, and D. Hu, “A multisynaptic spiking neuron for simultaneously encoding spatiotemporal dynamics,” *Nature Communications*, vol. 16, no. 1, p. 7155, 2025.
- [35] X. Xing, B. Gao, Z. Liu, D. A. Clifton, S. Xiao, W. Zhang, L. Du, Z. Zhang, G. Li, and J. Zhang, “Spikellm: Scaling up spiking neural network to large language models via saliency-based spiking,” in *Proceedings of the International Conference on Learning Representations (ICLR) 2025*, 2025.
- [36] S. Roy and A. Basu, “An online unsupervised structural plasticity algorithm for spiking neural networks,” *IEEE Transactions on Neural Networks and Learning Systems*, vol. 28, no. 4, pp. 900–910, 2017.
- [37] F. Paredes-Vallés, K. Y. W. Scheper, and G. C. H. E. de Croon, “Unsupervised learning of a hierarchical spiking neural network for optical flow estimation: From events to global motion perception,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 8, pp. 2051–2064, 2020.
- [38] T. Barbier, C. Teuliere, and J. Triesch, “Spike timing-based unsupervised learning of orientation, disparity, and motion representations in a spiking neural network,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, June 2021, pp. 1377–1386.
- [39] S. D. Pokala, M. Bernert, T. Nanami, T. Kohno, T. Lévi, and B. Yvert, “A frugal spiking neural network for unsupervised multivariate temporal pattern classification and multichannel spike sorting,” *Nature Communications*, vol. 16, no. 1, p. 64231, 2025.
- [40] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, “A fast and elitist multiobjective genetic algorithm: Nsga-ii,” *IEEE Transactions on Evolutionary Computation*, vol. 6, no. 2, pp. 182–197, 2002.
- [41] M. Osório, L. Sa-Couto, and A. Wichert, “Can a hebbian-like learning rule be avoiding the curse of dimensionality in sparse distributed data?” *Biological Cybernetics*, vol. 118, pp. 267–276, 2024.
- [42] G. G. Turrigiano, K. R. Leslie, N. S. Desai, L. C. Rutherford, and S. B. Nelson, “Activity-dependent scaling of quantal amplitude in neocortical neurons,” *Nature*, vol. 391, no. 6670, pp. 892–896, 1998.
- [43] G. Bellec, D. Salaj, A. Subramoney, R. Legenstein, and W. Maass, “Long short-term memory and learning-to-learn in networks of spiking neurons,” in *Advances in Neural Information Processing Systems*, vol. 31. Curran Associates, Inc., 2018.
- [44] N. Rathi and K. Roy, “Diet-snn: A low-latency spiking neural network with direct input encoding and leakage and threshold optimization,” *IEEE Transactions on Neural Networks and Learning Systems*, vol. 34, no. 6, pp. 3174–3182, 2021.
- [45] S. Qi, R. Wang, T. Zhang, X. Yang, R. Sun, and L. Wang, “A two-layer encoding learning swarm optimizer based on frequent itemsets for sparse large-scale multi-objective optimization,” *IEEE/CAA Journal of Automatica Sinica*, vol. 11, no. 6, pp. 1342–1357, 2024.
- [46] X. Wang, R. Cheng, and Y. Jin, “Sparse large-scale multiobjective optimization by identifying nonzero decision variables,” *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, vol. 54, no. 10, pp. 6280–6292, 2024.
- [47] I. Kropp, A. P. Nejadhashemi, and K. Deb, “Improved evolutionary operators for sparse large-scale multiobjective optimization problems,” *IEEE Transactions on Evolutionary Computation*, vol. 28, no. 2, pp. 460–473, 2024.
- [48] J. Jiang, F. Han, J. Wang, Q. Ling, H. Han, and Y. Wang, “A two-stage evolutionary algorithm for large-scale sparse multiobjective optimization problems,” *Swarm and Evolutionary Computation*, vol. 72, p. 101093, 2022.
- [49] Y. Li, X. Feng, and H. Yu, “A dynamic knowledge-guided coevolutionary algorithm for large-scale sparse multiobjective optimization problems,” *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, vol. 54, no. 11, pp. 7054–7064, 2024.
- [50] K. Deb, R. B. Agrawal *et al.*, “Simulated binary crossover for continuous search space,” *Complex systems*, vol. 9, no. 2, pp. 115–148, 1995.
- [51] E. Zitzler and L. Thiele, “Multiobjective evolutionary algorithms: a comparative case study and the strength pareto approach,” *IEEE Transactions on Evolutionary Computation*, vol. 3, no. 4, pp. 257–271, 1999.
- [52] P. He, R. Xiao, C. Tang, S. Huang, J. Lv, and H. Tang, “Stsf: Spiking time sparse feedback learning for spiking neural networks,” *IEEE Transactions on Neural Networks and Learning Systems*, vol. 36, no. 6, pp. 11 479–11 492, 2025.
- [53] B. Mészáros, J. C. Knight, and T. Nowotny, “Efficient event-based delay learning in spiking neural networks,” *Nature Communications*, vol. 16, p. 10422, 2025.



Fei Ming (Member IEEE) received the B.Sc. degree and Ph.D. degree from China University of Geosciences in 2019 and 2025, respectively. He was also a visiting Ph.D. student at Victoria University of Wellington, New Zealand. He is currently a postdoctoral researcher at Westlake University. His research interests include evolutionary multiobjective optimization methods and their applications. He has published over 40 research papers in international journals and conferences including IEEE Transactions on Evolutionary Computation, IEEE Transactions on Cybernetics, IEEE Computational Intelligence Magazine, IEEE/CAA Journal of Automatica Sinica, IEEE Transactions on Artificial Intelligence, ACM Transactions on Evolutionary Learning, and ICML. He received the IEEE Computational Intelligence Magazine Outstanding Paper Award and was selected for the Excellent Doctoral Dissertation Incentive Program of the Chinese Society for Simulation in 2026. He is the recipient of the Zhejiang Province Postdoctoral Research Project Excellence Funding and the Doctoral Student Special Program of the China Association for Science and Technology (CAST) Young Talent Support Program. He serves as a reviewer for over 30 international journals and conferences. He currently serves as Associate Editor of Complex & Intelligent Systems.



Xinming Shi (Member IEEE) received the B.Eng. degree in electronic engineering from Wuhan University of Technology, China, in 2016, the M.Eng. degree in control science and engineering from Huazhong University of Science and Technology, China, in 2019, and the Ph.D. degree in computer science from the University of Birmingham, U.K., in 2023. She was also a visiting researcher with Victoria University of Wellington, New Zealand. She is currently a Lecturer with the School of Electronics, Electrical Engineering and Computer Science, Queen's University Belfast, U.K. Her research interests include brain-inspired computing, memristive computing, evolutionary learning, evolvable hardware, and low-power intelligent systems. Her work has been published in leading IEEE and ACM journals, including IEEE Transactions on Neural Networks and Learning Systems, IEEE Transactions on Computers, and ACM Transactions on Design Automation of Electronic Systems. Dr. Shi is a recipient of the Leverhulme Early Career Fellowship and the IEEE Computational Intelligence Society Graduate Student Research Grant. She currently serves as an Associate Editor of IEEE Transactions on Cognitive and Developmental Systems and contributes to IEEE technical committees and conferences.



Wenxuan Pan (Member IEEE) received her B.Eng. degree from the University of Electronic Science and Technology of China, Chengdu, China. She received her Ph.D. degree from the Institute of Automation, Chinese Academy of Sciences, Beijing, China, under the supervision of Prof. Yi Zeng. She is currently a Postdoctoral Researcher at Westlake University, Hangzhou, China, working with Prof. Yaochu Jin. Her research interests focus on brain-inspired spiking neural networks.



Yaochu Jin (Fellow IEEE) received the BSc, MSc and PhD degrees from the Electrical Engineering Department, Zhejiang University, Hangzhou, China in 1988, 1991 and 1996, respectively. He received the Dr.-Ing. from the Institute of Neuroinformatics, Ruhr University Bochum, Germany in 2001. He is presently Chair Professor of AI, Head of the Artificial Intelligence Department, Director of the Trustworthy and General AI Laboratory, School of Engineering, Westlake University, Hangzhou, China.

His main research interests include AI theory, algorithms and applications to a wide range of scientific, technological, and industrial problems. Prof Jin was the President of the IEEE Computational Intelligence Society and is the Editor-in-Chief of *Complex & Intelligent Systems*. He is the recipient of the 2025 IEEE Frank Rosenblatt Award. He was named “Highly Cited Researcher” by Clarivate from 2019 consecutively. He is a Member of Academia Europaea and Fellow of IEEE.